# Monitoring & Logging Guide — Nucleus AI

Comprehensive guide for setting up monitoring, logging, and alerting.

---

## Health Check Endpoints

| Endpoint | Service | Expected Response |
|---|---|---|
| `/health` | Backend | `{"status": "healthy"}` |
| `/api/v1/health` | Backend | `{"status": "healthy"}` |
| `/api/health` | Frontend | Next.js health route |
| `:6333/readyz` | Qdrant | `200 OK` |

---

## 1. Structured Logging (Backend)

The backend uses `structlog` for structured JSON logging. Add to `backend/core/config.py`:

```
import structlog
import logging

def configure_logging(log_level: str = "INFO", log_format: str = "json"):
    """Configure structured logging for production."""
    processors = [
        structlog.contextvars.merge_contextvars,
        structlog.processors.add_log_level,
        structlog.processors.StackInfoRenderer(),
        structlog.processors.TimeStamper(fmt="iso"),
        structlog.processors.format_exc_info,
    ]

    if log_format == "json":
        processors.append(structlog.processors.JSONRenderer())
    else:
        processors.append(structlog.dev.ConsoleRenderer())

    structlog.configure(
        processors=processors,
        wrapper_class=structlog.make_filtering_bound_logger(
            getattr(logging, log_level.upper())
        ),
        context_class=dict,
        logger_factory=structlog.PrintLoggerFactory(),
        cache_logger_on_first_use=True,
    )
```

### Usage in Code

```
import structlog
logger = structlog.get_logger()

logger.info("request_processed",
    method="POST",
    path="/api/v1/context/ingest",
    duration_ms=245,
    brand_id="abc-123"
)
```

---

## 2. Sentry Integration (Error Tracking)

### Backend Setup

```
pip install sentry-sdk[fastapi]
```

```
# In main.py
import sentry_sdk
from sentry_sdk.integrations.fastapi import FastApiIntegration
from sentry_sdk.integrations.sqlalchemy import SqlalchemyIntegration

if settings.SENTRY_DSN:
    sentry_sdk.init(
        dsn=settings.SENTRY_DSN,
        environment=settings.APP_ENV,
        traces_sample_rate=0.1,  # 10% of transactions
        profiles_sample_rate=0.1,
        integrations=[
            FastApiIntegration(transaction_style="endpoint"),
            SqlalchemyIntegration(),
        ],
    )
```

### Frontend Setup

```
npm install @sentry/nextjs
npx @sentry/wizard@latest -i nextjs
```

---

## 3. Prometheus + Grafana (Self-Hosted Metrics)

### Add to docker-compose.prod.yml

```yaml
services:
  prometheus:
    image: prom/prometheus:v2.50.0
    restart: always
    volumes:
      - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prometheus_data:/prometheus
    ports:
      - "127.0.0.1:9090:9090"
    networks:
      - nucleus-net

  grafana:
    image: grafana/grafana:10.3.1
    restart: always
    environment:
      GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_PASSWORD:-admin}
      GF_USERS_ALLOW_SIGN_UP: "false"
    volumes:
      - grafana_data:/var/lib/grafana
    ports:
      - "127.0.0.1:3001:3000"
    depends_on:
      - prometheus
    networks:
      - nucleus-net

volumes:
  prometheus_data:
  grafana_data:
```

### Prometheus Configuration

```yaml
# monitoring/prometheus.yml
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'nucleus-backend'
    metrics_path: '/metrics'
    static_configs:
      - targets: ['backend:8000']

  - job_name: 'qdrant'
    metrics_path: '/metrics'
    static_configs:
      - targets: ['qdrant:6333']

  - job_name: 'nginx'
    static_configs:
      - targets: ['nginx-exporter:9113']

  - job_name: 'postgres'
    static_configs:
      - targets: ['postgres-exporter:9187']
```

### Add Prometheus Metrics to Backend

```
pip install prometheus-fastapi-instrumentator
```

```python
# In main.py
from prometheus_fastapi_instrumentator import Instrumentator

Instrumentator().instrument(app).expose(app)
```

---

## 4. Key Metrics to Monitor

### Application Metrics

| Metric | Alert Threshold | Description |
|---|---|---|
| HTTP request latency (p95) | > 2s | Slow API responses |
| HTTP 5xx rate | > 1% | Server errors |
| HTTP 4xx rate | > 10% | Client errors (may indicate attacks) |
| Active connections | > 1000 | Connection saturation |

### Infrastructure Metrics

| Metric | Alert Threshold | Description |
|---|---|---|
| CPU usage | > 80% for 5min | CPU saturation |
| Memory usage | > 85% | Memory pressure |
| Disk usage | > 80% | Disk running low |
| DB connection pool | > 80% used | Connection exhaustion |

### Business Metrics

| Metric | Description |
|---|---|
| Workflow executions/min | AI workflow throughput |
| Context ingestion rate | Documents processed |
| Active users | Current logged-in users |
| API key usage | Per-key request counts |

---

## 5. Log Aggregation

### For Docker Compose Deployments

```yaml
# Add to docker-compose.prod.yml
services:
  loki:
    image: grafana/loki:2.9.4
    volumes:
      - loki_data:/loki
    ports:
      - "127.0.0.1:3100:3100"
    networks:
      - nucleus-net

  promtail:
    image: grafana/promtail:2.9.4
    volumes:
      - /var/log:/var/log:ro
      - /var/lib/docker/containers:/var/lib/docker/containers:ro
      - ./monitoring/promtail.yml:/etc/promtail/config.yml:ro
    networks:
      - nucleus-net
```

### For Cloud Deployments

- **AWS:** CloudWatch Logs (automatic with ECS)
- **GCP:** Cloud Logging (automatic with Cloud Run)

- **Azure:** Azure Monitor Logs

---

## 6. Uptime Monitoring

Recommended external monitoring services:
- **UptimeRobot** (free tier: 50 monitors)
- **Better Uptime** (free tier available)
- **Pingdom**

Monitor these URLs:
1. `https://yourdomain.com` — Frontend
2. `https://yourdomain.com/health` — Backend health
3. `https://yourdomain.com/api/v1/health` — API health